Code:

```
#include <fstream>
#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/internet-module.h"
#include "ns3/point-to-point-module.h"
#include "ns3/applications-module.h"
#include "ns3/stats-module.h"
#include "ns3/gnuplot.h"

using namespace ns3;

NS_LOG_COMPONENT_DEFINE ("SeventhScriptExample");

class MyApp : public Application
{
public:
  MyApp ();
  virtual ~MyApp ();
  static TypeId GetTypeId (void);

  void Setup (Ptr<Socket> socket, Address address,
            uint32_t packetSize, uint32_t nPackets,
            DataRate dataRate);

private:
  virtual void StartApplication (void);
  virtual void StopApplication (void);

  void ScheduleTx (void);
  void SendPacket (void);

  Ptr<Socket>     m_socket;
```

```
  Address         m_peer;
  uint32_t        m_packetSize;
  uint32_t        m_nPackets;
  DataRate        m_dataRate;
  EventId         m_sendEvent;
  bool            m_running;
  uint32_t        m_packetsSent;
};

MyApp::MyApp ()
  : m_socket (0),
    m_peer (),
    m_packetSize (0),
    m_nPackets (0),
    m_dataRate (0),
    m_running (false),
    m_packetsSent (0)
{
}

MyApp::~MyApp ()
{
  m_socket = 0;
}

TypeId
MyApp::GetTypeId (void)
{
  static TypeId tid = TypeId ("MyApp")
    .SetParent<Application> ()
    .SetGroupName ("Tutorial")
    .AddConstructor<MyApp> ();
  return tid;
}
```

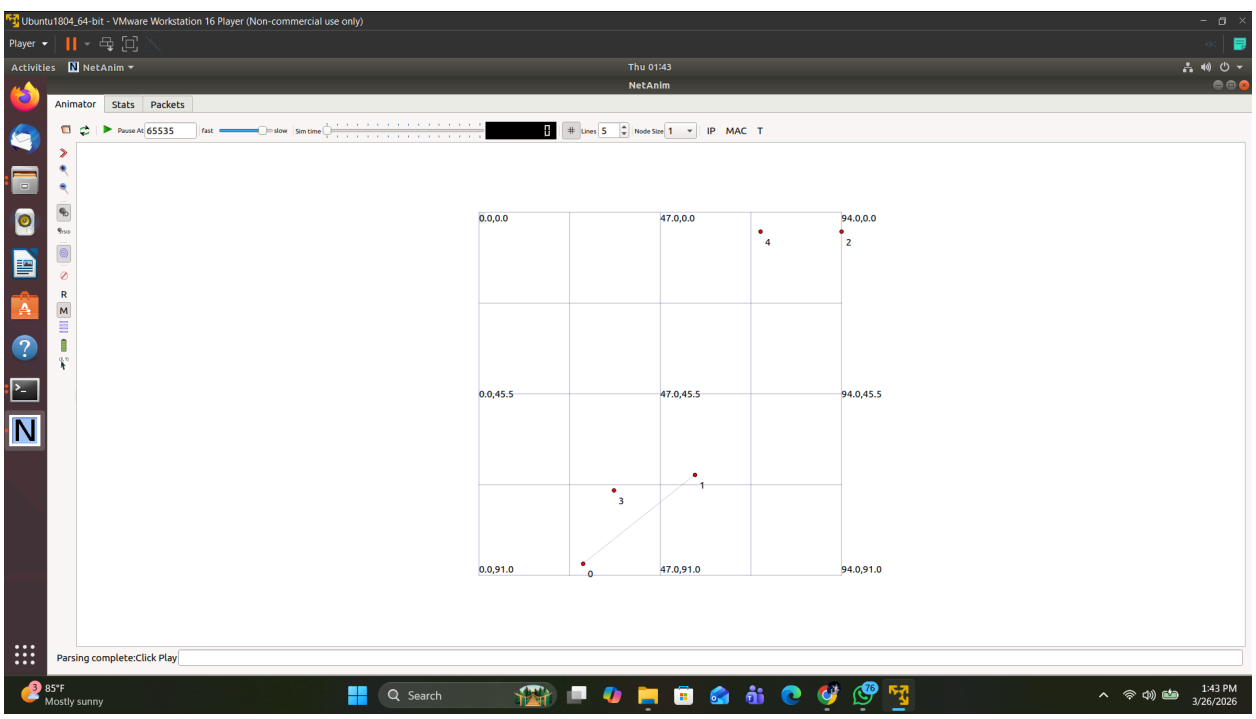
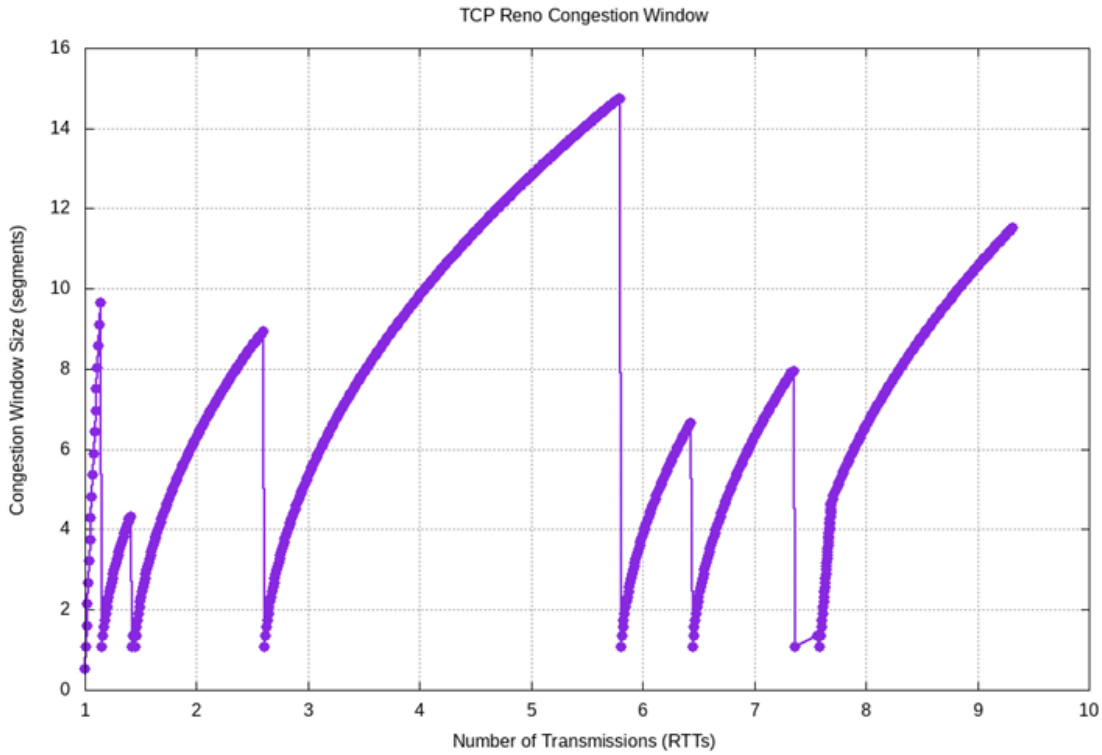
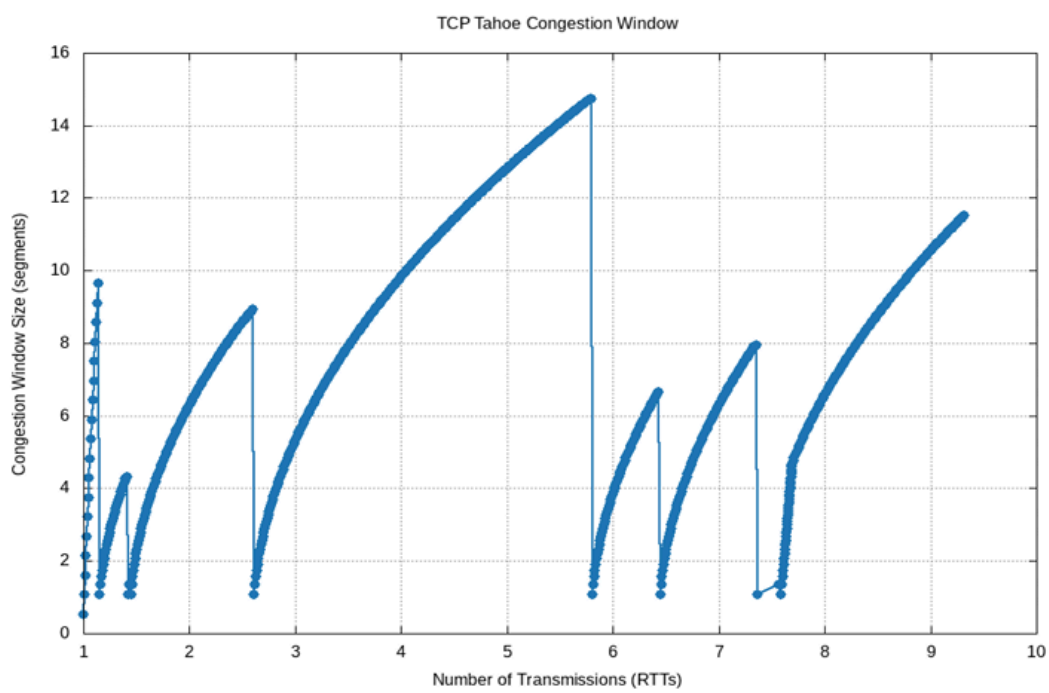
```
void
MyApp::Setup (Ptr<Socket> socket, Address address,
            uint32_t packetSize, uint32_t nPackets,
            DataRate dataRate)
{
  m_socket = socket;
  m_peer = address;
  m_packetSize = packetSize;
  m_nPackets = nPackets;
  m_dataRate = dataRate;
}

void
MyApp::StartApplication (void)
{
  m_running = true;
  m_packetsSent = 0;

  if (InetSocketAddress::IsMatchingType (m_peer))
    m_socket->Bind ();
  else
    m_socket->Bind6 ();

  m_socket->Connect (m_peer);
  SendPacket ();
}

void
MyApp::StopApplication (void)
{
  m_running = false;

  if (m_sendEvent.IsRunning ())
    Simulator::Cancel (m_sendEvent);
```

```
  if (m_socket)
    m_socket->Close ();
}

void
MyApp::SendPacket (void)
{
  Ptr<Packet> packet = Create<Packet> (m_packetSize);
  m_socket->Send (packet);

  if (++m_packetsSent < m_nPackets)
    ScheduleTx ();
}

void
MyApp::ScheduleTx (void)
{
  if (m_running)
  {
    Time tNext (Seconds (m_packetSize * 8 /
      static_cast<double> (m_dataRate.GetBitRate ())));
    m_sendEvent = Simulator::Schedule (tNext,
      &MyApp::SendPacket, this);
  }
}

static Gnuplot2dDataset cwndDataset;

static void
CwndChange (Ptr<OutputStreamWrapper> stream,
          uint32_t oldCwnd, uint32_t newCwnd)
{
  double time = Simulator::Now ().GetSeconds ();
  *stream->GetStream () << time << "\t"
                    << oldCwnd << "\t"
```

```
                    << newCwnd << std::endl;
  cwndDataset.Add (time, newCwnd);
}

static void
RxDrop (Ptr<PcapFileWrapper> file, Ptr<const Packet> p)
{
  file->Write (Simulator::Now (), p);
}

int
main (int argc, char *argv[])
{
  bool useV6 = false;
  CommandLine cmd;
  cmd.AddValue ("useIpv6", "Use Ipv6", useV6);
  cmd.Parse (argc, argv);

  NodeContainer nodes;
  nodes.Create (2);

  PointToPointHelper pointToPoint;
  pointToPoint.SetDeviceAttribute ("DataRate", StringValue
("5Mbps"));
  pointToPoint.SetChannelAttribute ("Delay", StringValue ("2ms"));

  NetDeviceContainer devices = pointToPoint.Install (nodes);

  Ptr<RateErrorModel> em = CreateObject<RateErrorModel> ();
  em->SetAttribute ("ErrorRate", DoubleValue (0.00001));
  devices.Get (1)->SetAttribute ("ReceiveErrorModel", PointerValue
(em));

  InternetStackHelper stack;
  stack.Install (nodes);
```

```
  uint16_t sinkPort = 8080;
  Address sinkAddress, anyAddress;
  std::string probeType, tracePath;

  if (!useV6)
  {
    Ipv4AddressHelper address;
    address.SetBase ("10.1.1.0", "255.255.255.0");
    auto interfaces = address.Assign (devices);
    sinkAddress = InetSocketAddress (interfaces.GetAddress (1),
sinkPort);
    anyAddress  = InetSocketAddress (Ipv4Address::GetAny (),
sinkPort);
    probeType = "ns3::Ipv4PacketProbe";
    tracePath = "/NodeList/*/$ns3::Ipv4L3Protocol/Tx";
  }
  else
  {
    Ipv6AddressHelper address;
    address.SetBase ("2001:0000:f00d:cafe::", Ipv6Prefix (64));
    auto interfaces = address.Assign (devices);
    sinkAddress = Inet6SocketAddress (interfaces.GetAddress (1,1),
sinkPort);
    anyAddress  = Inet6SocketAddress (Ipv6Address::GetAny (),
sinkPort);
    probeType = "ns3::Ipv6PacketProbe";
    tracePath = "/NodeList/*/$ns3::Ipv6L3Protocol/Tx";
  }

  PacketSinkHelper sink ("ns3::TcpSocketFactory", anyAddress);
  sink.Install (nodes.Get (1))->Start (Seconds (0));

  Ptr<Socket> socket =
    Socket::CreateSocket (nodes.Get (0),
```

```
  TcpSocketFactory::GetTypeId ());

  Ptr<MyApp> app = CreateObject<MyApp> ();
  app->Setup (socket, sinkAddress, 1040, 1000, DataRate ("1Mbps"));
  nodes.Get (0)->AddApplication (app);
  app->SetStartTime (Seconds (1));
  app->SetStopTime (Seconds (20));

  Simulator::Stop (Seconds (20));
  Simulator::Run ();
  Simulator::Destroy ();
  return 0;
}
```



Congestion Window Graph